

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
CST ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PROJETO: DESENVOLVIMENTO BACK-END

GEAN CARLOS BRANDÃO - 4836754

API REST PARA REDE DE LANCHONETES MULTIUNIDADE E MULTICANAL:
O CASO RAÍZES DO NORDESTE

BELO HORIZONTE - MG

2026

SUMÁRIO

1 INTRODUÇÃO	2
1.1 Objetivo geral	2
1.2 Objetivos específicos	2
1.3 Usuários do sistema	3
1.4 Relevância.....	3
2 ANÁLISE E REQUISITOS	4
2.1 Requisitos funcionais	4
2.2 Regras de negócio do fluxo crítico	5
2.3 Requisitos não funcionais	5
2.4 Priorização: implementado e proposto	6
3 MODELAGEM E DIAGRAMAS.....	8
3.1 Diagrama de casos de uso.....	8
3.2 Descrição da feature: realizar pedido e solicitar pagamento	10
3.3 Modelo de dados.....	14
3.4 Diagrama de classes.....	16
3.5 Diagrama de sequência do fluxo crítico	18
4 ARQUITETURA	20
4.1 Organização em camadas	20
4.2 Direção das dependências.....	20
4.3 Localização das regras de negócio.....	21
4.4 Portas para integrações externas	22
5 API E ENDPOINTS	23
5.1 Visão geral por recurso	23
5.2 Convenções do contrato.....	23
5.3 Detalhamento das operações do fluxo crítico	24
5.4 Padrão de erro	26

5.5 Documentação interativa e coleção de testes.....	28
6 LGPD, PRIVACIDADE E SEGURANÇA.....	29
6.1 Dados pessoais tratados	29
6.2 Consentimento e revogação	29
6.3 Controles técnicos.....	30
6.4 Decisões de segurança	30
6.5 Trilha de auditoria.....	31
6.6 Limitações conhecidas.....	32
7 ENTREGA TÉCNICA	33
7.1 Repositório.....	33
7.2 Requisitos e execução	33
7.3 Documentação da API	34
7.4 Coleção de testes.....	34
7.5 Evidências	34
8 PLANO DE TESTES	36
8.1 Cenários	36
8.2 Cobertura	37
8.3 Testes automatizados	38
8.4 Resultados.....	38
9 CONCLUSÃO	39

1 INTRODUÇÃO

A Raízes do Nordeste é uma rede de lanchonetes especializada em comida típica nordestina — tapioca, cuscuz, bolo de rolo, canjica — com unidades em cidades diferentes de Pernambuco e venda por múltiplos canais: aplicativo, site, totem de autoatendimento e balcão. O crescimento da rede trouxe um problema que o controle manual já não resolve: cada unidade opera com cardápio e preço próprios, o estoque é local, as campanhas promocionais variam por canal e por público, e a operação precisa saber, a qualquer momento, o que foi vendido, por onde e por quem.

Este trabalho apresenta o desenvolvimento do back-end que sustenta essa operação: uma API REST responsável pelo ciclo completo do pedido, do cardápio da unidade até a confirmação do pagamento e o acompanhamento do preparo.

1.1 Objetivo geral

Projetar e implementar uma API REST que atenda à operação multiunidade e multicanal da rede, garantindo consistência de estoque, rastreabilidade das ações sensíveis e conformidade com a Lei Geral de Proteção de Dados.

1.2 Objetivos específicos

- a) Modelar o domínio da rede, contemplando unidades, catálogo de produtos, cardápio por unidade, estoque, pedidos, pagamentos, campanhas promocionais e programa de fidelidade;
- b) Implementar autenticação por token e autorização por perfil, com escopo de acesso restrito à unidade e ao titular dos dados;
- c) Assegurar que a criação do pedido seja atômica em relação ao estoque, impedindo venda sem saldo;
- d) Integrar um gateway de pagamento simulado com desfechos determinísticos, incluindo recusa e ausência de resposta;
- e) Registrar as ações sensíveis em trilha de auditoria não adulterável pela API;
- f) Implementar os controles de LGPD aplicáveis: consentimento versionado por finalidade, revogação e minimização de dados nas respostas;
- g) Cobrir o comportamento esperado com testes automatizados e disponibilizar coleção reproduzível para verificação por terceiros.

1.3 Usuários do sistema

O sistema atende cinco perfis internos e um ator externo, descritos no Quadro 1.

Quadro 1 – Perfis de usuário e responsabilidades

Perfil	Responsabilidade
Cliente	Consulta o cardápio, realiza e acompanha pedidos, administra seus pontos e seus consentimentos
Atendente	Registra pedidos de balcão e acompanha o andamento
Cozinha	Atualiza o status de preparo
Gerente	Administra cardápio e estoque da própria unidade e acompanha os pedidos dela
Administrador	Administra produtos, unidades e campanhas da rede e consulta a trilha de auditoria

Fonte: Elaborado pelo autor (2026).

Há ainda um ator externo: o gateway de pagamento, que recebe a solicitação de cobrança e pode confirmar o resultado posteriormente por callback.

1.4 Relevância

O problema concentra três dificuldades recorrentes em sistemas comerciais reais, e é essa combinação que justifica o recorte.

A primeira é a consistência sob concorrência: duas pessoas podem pedir o último item ao mesmo tempo, e o sistema precisa decidir sem vender o que não tem.

A segunda é a variação por unidade: tratar a rede como uma loja só seria mais simples de implementar e erraria o negócio, porque preço, disponibilidade e estoque são locais.

A terceira é a prestação de contas: franquia exige saber quem cancelou um pedido, quem alterou um preço e quem ajustou um estoque — e exige que esse registro não possa ser alterado por quem é auditado.

A LGPD atravessa as três. O programa de fidelidade depende de identificar o cliente e conhecer seu consumo, o que caracteriza tratamento de dado pessoal e exige base legal explícita. A decisão de projeto adotada foi condicionar a pontuação ao consentimento ativo, assumindo que o cliente sem consentimento simplesmente não acumula pontos.

2 ANÁLISE E REQUISITOS

Os requisitos foram levantados a partir do estudo de caso com foco em regras de negócio, persistência, integrações e segurança. Questões de interface não são tratadas, por estarem fora do escopo de um projeto de back-end.

2.1 Requisitos funcionais

O Quadro 2 apresenta os requisitos funcionais identificados.

Quadro 2 – Requisitos funcionais

Código	Requisito
RF01	Cadastro de usuário com e-mail e senha
RF02	Autenticação com emissão de token de acesso e token de renovação
RF03	Perfis de acesso com autorização por endpoint
RF04	Gestão das unidades da rede
RF05	Catálogo de produtos da rede
RF06	Cardápio por unidade, com preço e disponibilidade locais
RF07	Consulta pública do cardápio de uma unidade
RF08	Criação de pedido com itens, canal, valores e status
RF09	Cálculo do total do pedido no servidor
RF10	Registro obrigatório do canal de origem do pedido
RF11	Consulta e filtro de pedidos por canal, status e unidade
RF12	Atualização do status do pedido conforme máquina de estados
RF13	Cancelamento de pedido com devolução do estoque
RF14	Controle de saldo de estoque por unidade
RF15	Movimentação de estoque com histórico
RF16	Bloqueio da venda quando não há saldo disponível
RF17	Solicitação de pagamento a serviço externo simulado
RF18	Registro do retorno do pagamento
RF19	Atualização do status do pedido conforme o retorno do pagamento
RF20	Acúmulo de pontos de fidelidade em pedido pago
RF21	Consulta de saldo e extrato de pontos
RF22	Resgate de pontos
RF23	Consentimento explícito para participação no programa de fidelidade
RF24	Revogação do consentimento
RF25	Registro de auditoria das ações sensíveis
RF26	Consulta da trilha de auditoria pelo perfil administrador
RF27	Campanhas promocionais com aplicação automática de desconto
RF28	Segmentação de campanha por canal, unidade e faixa etária
RF29	Relatórios consolidados de vendas para a matriz

Código	Requisito
RF30	Produtos sazonais com janela de vigência própria
RF31	Descontos progressivos por frequência de compra
RF32	Anonimização de dados pessoais após o prazo de retenção

Fonte: Elaborado pelo autor (2026).

2.2 Regras de negócio do fluxo crítico

O fluxo de pedido e pagamento concentra as regras que exigem maior rigor, por envolverem valor financeiro e disponibilidade física de produto. O Quadro 3 as apresenta.

Quadro 3 – Regras de negócio do fluxo crítico

Código	Regra
RN01	O pedido só aceita produtos presentes no cardápio daquela unidade e ativos
RN02	O preço unitário é congelado no item no momento da criação do pedido
RN03	O total é calculado no servidor como a soma dos subtotais menos o desconto
RN04	O estoque é debitado na criação do pedido; insuficiência impede a criação
RN05	Canal de pedido ausente ou inválido impede a criação, com erro padronizado
RN06	O pedido nasce aguardando pagamento
RN07	Pagamento aprovado leva o pedido a pago; recusado, a pagamento recusado
RN08	A transição de status só ocorre pelos caminhos válidos da máquina de estados
RN09	O cancelamento só é permitido antes do preparo estar pronto e devolve o estoque
RN10	Pontos de fidelidade só acumulam em pedido pago e com consentimento ativo
RN11	O resgate de pontos exige saldo suficiente
RN12	O cliente enxerga apenas os próprios pedidos; o operador, apenas os da sua unidade
RN13	Toda ação sensível gera registro de auditoria na mesma transação da ação
RN14	A solicitação de pagamento é idempotente: o reenvio não duplica a cobrança

Fonte: Elaborado pelo autor (2026).

2.3 Requisitos não funcionais

Os requisitos não funcionais e as soluções adotadas para cada um constam do Quadro 4.

Quadro 4 – Requisitos não funcionais

Código	Requisito	Solução adotada
RNF01	Armazenamento seguro de senha	Hash BCrypt, com sal próprio por senha
RNF02	Autenticação por token	Token de acesso com validade de 15 minutos e token de renovação com rotação

Código	Requisito	Solução adotada
RNF03	Autorização por perfil	Controle por endpoint, com negação por padrão
RNF04	Não exposição de dados sensíveis	Objetos de resposta dedicados; a entidade nunca é devolvida
RNF05	Padrão de erro consistente	Tratador global de exceções, com envelope único
RNF06	Documentação da API	Especificação OpenAPI gerada a partir do código
RNF07	Auditoria de ações sensíveis	Tabela de trilha, gravada na mesma transação da ação
RNF08	Desempenho em horário de pico	Índices nas chaves de busca e paginação obrigatória nas listagens
RNF09	Tolerância a falha na integração de pagamento	Tempo limite, estado intermediário, confirmação por retorno assíncrono e idempotência
RNF10	Verificação de disponibilidade	Endpoint de saúde da aplicação
RNF11	Escalabilidade horizontal	API sem estado; o token carrega o contexto da requisição
RNF12	Retenção e anonimização de dados pessoais	Estratégia descrita, sem implementação

Fonte: Elaborado pelo autor (2026).

2.4 Priorização: implementado e proposto

Dos 32 requisitos funcionais levantados, 28 foram implementados e 4 permanecem como proposta. Entre os não funcionais, 10 foram implementados integralmente, 1 parcialmente e 1 permanece como proposta.

O critério de priorização foi o risco técnico. Foram implementados primeiro os requisitos que concentram as decisões difíceis — consistência de estoque sob concorrência, variação de preço entre unidades, idempotência de cobrança, base legal para tratamento de dado pessoal — porque são eles que determinam se a arquitetura se sustenta. Os requisitos adiados compartilham uma característica: todos reutilizam mecanismos já demonstrados, sem introduzir risco novo.

Quadro 5 – Requisitos adiados e justificativa

Código	Requisito	Por que ficou como proposta
RF29	Relatórios consolidados de vendas	Os dados já estão modelados e acessíveis: todo pedido registra unidade, canal e total, e as listagens aceitam filtro por esses campos. Construir endpoints de relatório acrescentaria superfície de API sem exercitar nenhuma decisão técnica nova
RF30	Produtos sazonais com janela de vigência	O catálogo já distingue o produto sazonal por atributo próprio. A janela de datas repetiria o mecanismo de vigência que as campanhas já implementam, com início e fim de validade
RF31	Descontos progressivos por frequência	O motor de campanhas já segmenta por canal, unidade e faixa etária. Frequência de compra seria um quarto critério do mesmo mecanismo, sem ganho de demonstração

Código	Requisito	Por que ficou como proposta
RF32 e RNF12	Anonimização após retenção	Depende de uma política de retenção que o caso fictício não define. Implementar um prazo arbitrário significaria inventar uma regra de negócio, o que seria pior do que declarar a lacuna
RNF08	Limitação de taxa de requisições	Implementado parcialmente. Os índices e a paginação obrigatória atendem o desempenho; a limitação por origem tem as bibliotecas declaradas no projeto, mas o filtro não foi conectado. Está registrado como pendência no repositório

Fonte: Elaborado pelo autor (2026).

Três requisitos merecem destaque por atravessarem todo o sistema em vez de se concentrarem em um módulo.

A multicanalidade não é um campo de registro, mas um dado de domínio com efeito prático: o canal é obrigatório na criação do pedido, permite filtrar as consultas e é um dos critérios de segmentação das campanhas. O mesmo pedido, na mesma unidade, resulta em valor diferente conforme o canal de origem.

A integração de pagamento simulada foi tratada como integração real, não como atalho. O serviço externo é representado por uma interface, e a implementação simulada produz os três desfechos que uma integração verdadeira produz: aprovação, recusa e ausência de resposta. Esse terceiro caso é o que justifica o estado intermediário e o retorno assíncrono de confirmação.

A LGPD condiciona uma funcionalidade do sistema. O programa de fidelidade depende de identificar o cliente e conhecer seu consumo, o que caracteriza tratamento de dado pessoal. A decisão foi condicionar a pontuação ao consentimento ativo: sem ele, o pedido é pago normalmente e simplesmente não gera pontos.

3 MODELAGEM E DIAGRAMAS

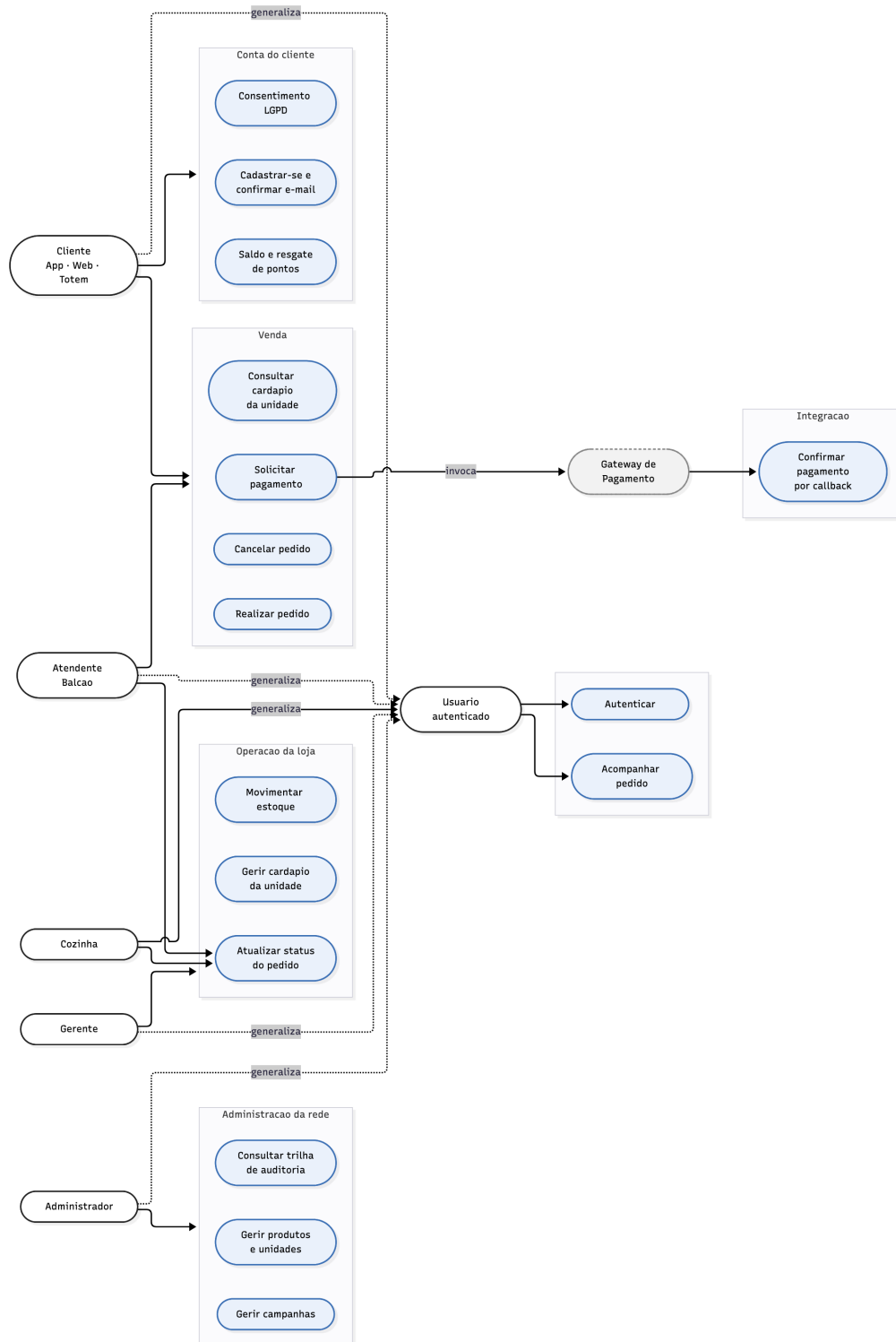
Esta seção apresenta a modelagem da solução em quatro visões complementares: os casos de uso, que delimitam o que cada ator pode fazer; a descrição detalhada da funcionalidade crítica; o modelo de dados; e as visões estrutural e comportamental do domínio.

3.1 Diagrama de casos de uso

A Figura 1 apresenta os casos de uso do sistema e os atores envolvidos. Cinco atores humanos são especializações de um ator genérico, o usuário autenticado, que detém os casos comuns a todos: autenticar-se e acompanhar pedidos. Essa generalização evita repetir as mesmas associações cinco vezes e deixa visível o que é específico de cada perfil.

O gateway de pagamento aparece como ator externo porque não é operado por pessoa alguma da rede: ele recebe a solicitação de cobrança e, quando não responde a tempo, confirma o resultado depois, por iniciativa própria. É o único caso de uso iniciado de fora do sistema.

Figura 1 – Diagrama de casos de uso



Fonte: Elaborado pelo autor (2026).

3.2 Descrição da feature: realizar pedido e solicitar pagamento

A funcionalidade crítica do sistema é a que vai da montagem do pedido à confirmação da cobrança, por concentrar valor financeiro e disponibilidade física de produto. O Quadro 6 a detalha no formato de descrição de caso de uso.

Quadro 6 – Descrição da feature realizar pedido e solicitar pagamento

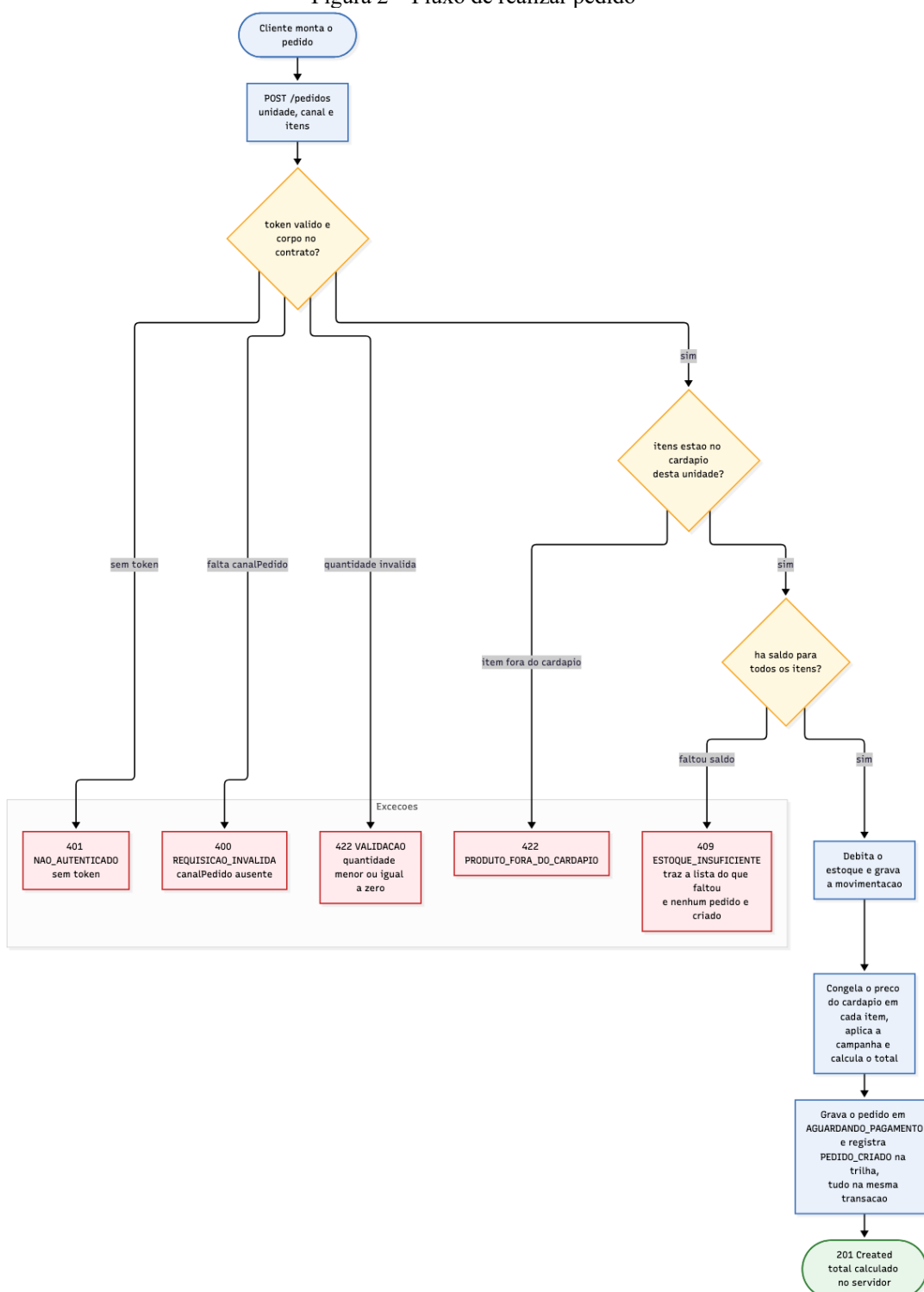
Campo	Conteúdo
Descrição	Permite ao cliente, ou ao atendente em seu nome, montar um pedido com itens do cardápio de uma unidade e solicitar a cobrança ao serviço externo de pagamento.
Ator principal	Cliente, pelos canais aplicativo, web e totem.
Demais atores	Atendente, no canal balcão. Gateway de pagamento, como sistema externo.
Pré-condições	1. O solicitante está autenticado e possui perfil de cliente ou de atendente; 2. A unidade escolhida está ativa; 3. Os produtos pedidos constam do cardápio daquela unidade e estão disponíveis; 4. Há saldo de estoque suficiente para todos os itens na unidade.
Pós-condições	1. O pedido está persistido com o canal de origem e o total calculado pelo servidor; 2. O estoque da unidade foi debitado e a movimentação, registrada com o saldo resultante; 3. A criação do pedido consta da trilha de auditoria; 4. Com o pagamento aprovado, o pedido está pago e os pontos foram creditados, se houver consentimento ativo.
Origem das informações	1. Cardápio da unidade, para o preço vigente de cada item; 2. Catálogo de produtos da rede, para a identificação do item; 3. Campanhas vigentes, para o desconto aplicável; 4. Saldo de estoque da unidade, para a disponibilidade.
Usuários responsáveis	Cliente e Atendente.
Pendências	Integração com provedor real de pagamento e limitação de taxa nos endpoints públicos.
Fluxo base	1. O ator consulta o cardápio da unidade escolhida; 2. O ator monta a lista de itens com as respectivas quantidades e informa o canal de origem; 3. O sistema valida o token e o perfil do solicitante; 4. O sistema confere se todos os itens constam do cardápio daquela unidade e estão disponíveis; 5. O sistema verifica o saldo de estoque de todos os itens de uma só vez; 6. O sistema debita o estoque e registra a movimentação com o saldo resultante; 7. O sistema lê o preço vigente de cada item no cardápio e o congela no pedido; 8. O sistema aplica a campanha vigente para aquele canal, aquela unidade e aquela faixa etária; 9. O sistema calcula o total e grava o pedido com status aguardando pagamento; 10. O sistema registra a criação na trilha de auditoria, na mesma transação; 11. O sistema devolve o pedido criado ao ator; 12. O ator solicita o pagamento, informando o método; 13. O sistema verifica se já existe pagamento para a mesma chave de idempotência; 14. O sistema solicita a cobrança ao serviço externo; 15. O serviço externo responde com aprovação; 16. O sistema altera o status do pedido para pago; 17. O sistema credita os pontos de fidelidade, se houver consentimento ativo; 18. O sistema registra a solicitação de pagamento na trilha de auditoria.

Campo	Conteúdo
Fluxo alternativo	A1. Pagamento recusado — no passo 15, o serviço externo recusa a cobrança. O sistema registra a recusa, altera o pedido para pagamento recusado e devolve o resultado. O ator pode solicitar o pagamento novamente, com outro método; A2. Serviço externo sem resposta — no passo 15, o serviço externo não responde dentro do tempo limite. O pagamento fica pendente e o pedido não avança. A confirmação chega depois, por retorno assíncrono autenticado por segredo compartilhado; A3. Reenvio da mesma solicitação — no passo 13, já existe pagamento para aquela chave. O sistema devolve o pagamento existente sem acionar o serviço externo.
Exceções	E1. Token ausente ou inválido: a requisição é recusada como não autenticada; E2. Canal de origem ausente ou inexistente: o corpo não atende ao contrato e a requisição é recusada; E3. Quantidade menor ou igual a zero: o valor é inválido e a requisição é recusada; E4. Item fora do cardápio da unidade: a requisição é recusada, identificando o item; E5. Saldo insuficiente: a requisição é recusada com a lista completa dos itens em falta, e nenhum pedido é criado; E6. Pedido já pago: a nova solicitação de pagamento é recusada.
Regras de negócio	RN01, RN02, RN03, RN04, RN05, RN06, RN07, RN13 e RN14, conforme o Quadro 3.

Fonte: Elaborado pelo autor (2026).

As Figuras 2 e 3 representam o mesmo fluxo em forma de diagrama, separadas em duas etapas para preservar a legibilidade. A Figura 2 cobre a criação do pedido e suas cinco saídas de erro.

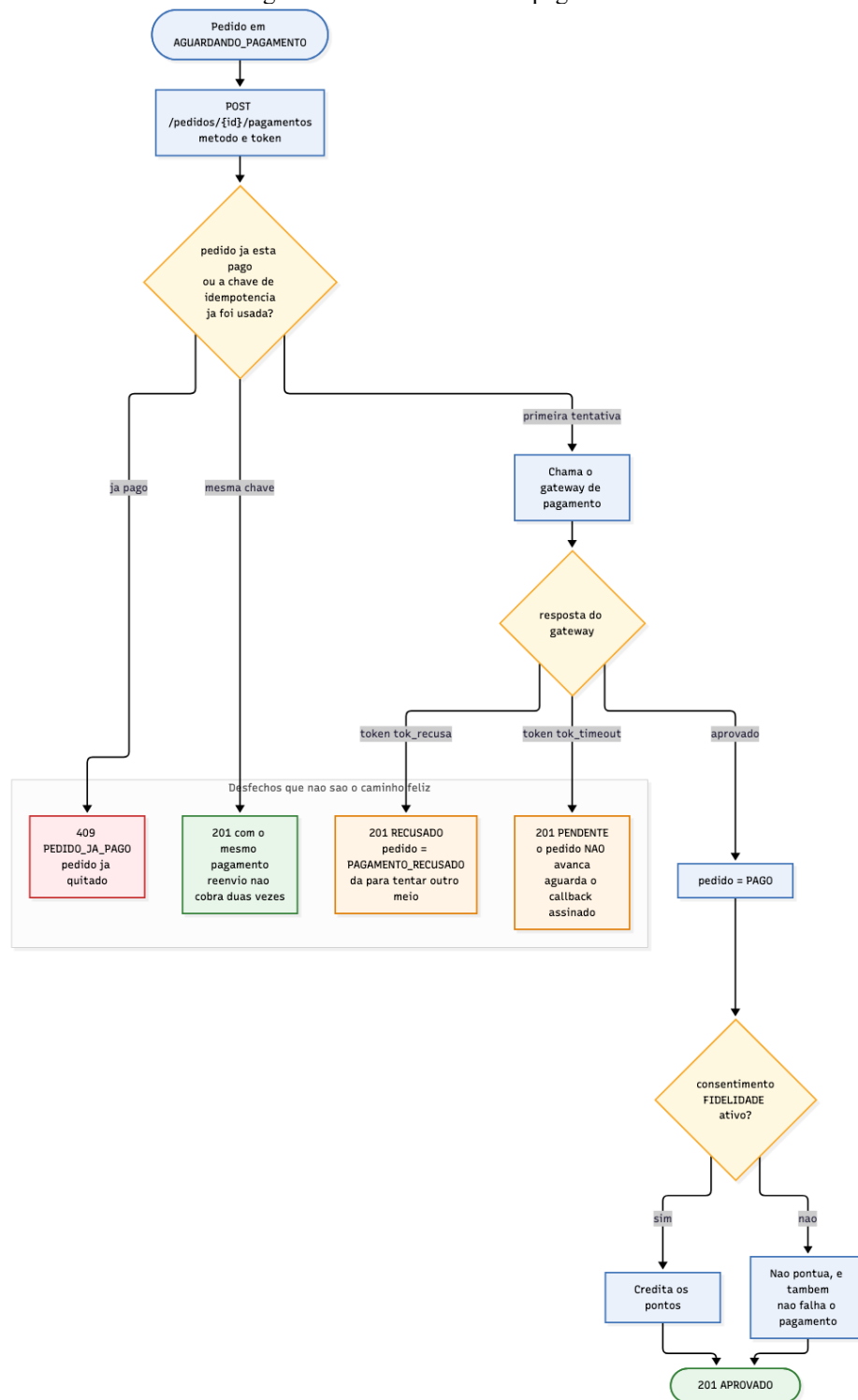
Figura 2 – Fluxo de realizar pedido



Fonte: Elaborado pelo autor (2026).

A Figura 3 cobre a solicitação de pagamento, com os três desfechos possíveis do serviço externo e o tratamento de reenvio.

Figura 3 – Fluxo de solicitar pagamento



Fonte: Elaborado pelo autor (2026).

3.3 Modelo de dados

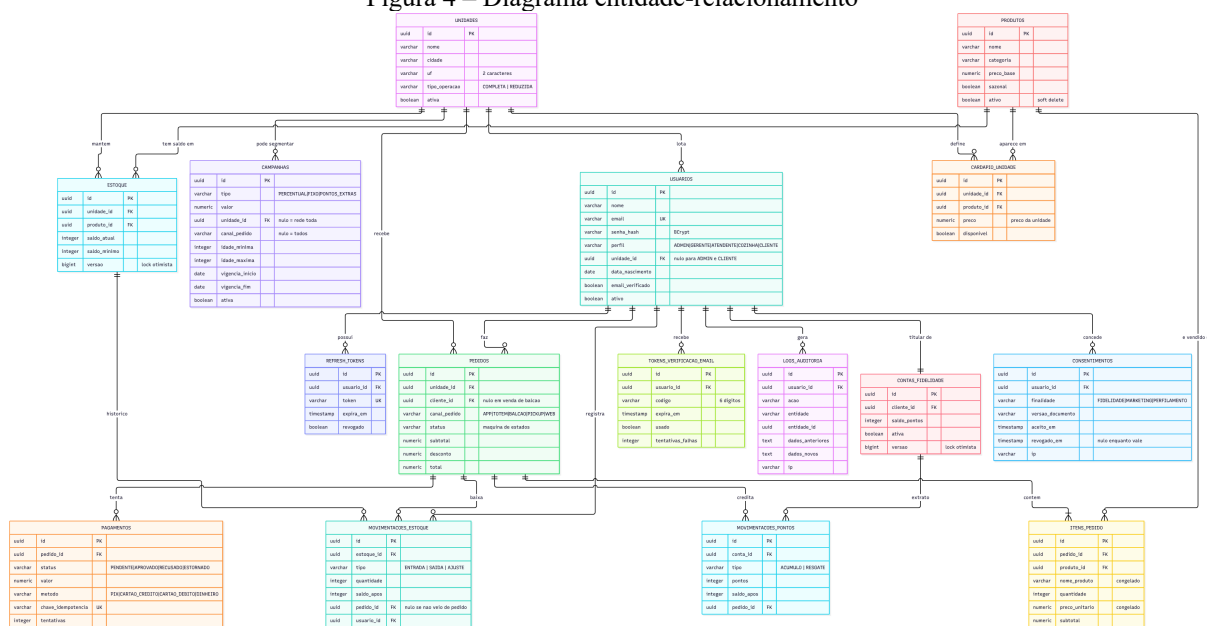
O modelo possui dezesseis tabelas, descritas no Quadro 7. A Figura 4 apresenta o diagrama entidade-relacionamento correspondente.

Quadro 7 – Entidades do modelo de dados

Tabela	Finalidade
unidades	Lojas da rede. Define o tipo de operação, completa ou reduzida.
usuarios	Pessoas com acesso ao sistema. O perfil define a autorização; a unidade vincula o operador à sua loja.
produtos	Catálogo da rede. A inativação é lógica, para não invalidar pedidos antigos.
cardapio_unidade	Associa produto e unidade com preço e disponibilidade locais. É o que torna as lojas diferentes entre si.
estoque	Saldo de um produto em uma unidade. Possui controle de concorrência otimista.
movimentacoes_estoque	Histórico de entradas, saídas e ajustes, com o saldo resultante de cada movimento.
pedidos	Pedido com unidade, cliente, canal de origem, status e valores.
itens_pedido	Itens do pedido, com nome e preço congelados no momento da criação.
pagamentos	Tentativas de cobrança de um pedido, com chave de idempotência única.
campanhas	Promoções segmentadas por unidade, canal e faixa etária, com janela de vigência.
contas_fidelidade	Conta de pontos do cliente. Possui controle de concorrência otimista.
movimentacoes_pontos	Extrato de acúmulos e resgates, com o saldo resultante.
consentimentos	Consentimento por finalidade, versionado. A revogação preserva a linha.
logs_auditoria	Trilha das ações sensíveis, com estado anterior e novo.
refresh_tokens	Tokens de renovação de sessão, com rotação.
tokens_verificacao_email	Códigos de confirmação de e-mail, com validade e limite de tentativas.

Fonte: Elaborado pelo autor (2026).

Figura 4 – Diagrama entidade-relacionamento



Fonte: Elaborado pelo autor (2026).

Três decisões de modelagem merecem explicação, por não serem a escolha mais imediata.

O cardápio é uma entidade própria, e não um atributo do produto. Se o preço vivesse no catálogo da rede, todas as lojas venderiam pelo mesmo valor, o que contradiz o estudo de caso: a unidade de operação reduzida pratica preço menor que a completa. A separação permite que o mesmo produto tenha preço e disponibilidade distintos em cada loja.

O item de pedido guarda cópia do nome e do preço, em vez de apenas apontar para o produto. É duplicação deliberada: o pedido precisa registrar quanto foi cobrado naquele momento, e um reajuste posterior no cardápio não pode alterar o valor de uma venda já realizada.

O pagamento é tabela separada do pedido, e não um conjunto de colunas nele. Um pedido pode ter várias tentativas de cobrança — uma recusada e outra aprovada, por exemplo — e cada tentativa precisa do próprio status, da própria chave de idempotência e do próprio retorno do serviço externo.

O Quadro 8 relaciona as restrições de integridade aplicadas.

Quadro 8 – Restrições de integridade do modelo

Tipo	Aplicação
Chave primária	Identificador universal único em todas as tabelas, gerado pela aplicação
Unicidade	E-mail do usuário; chave de idempotência do pagamento; token de renovação; par unidade e produto no cardápio e no estoque
Integridade referencial	Vinte chaves estrangeiras; o item de pedido não existe sem o pedido

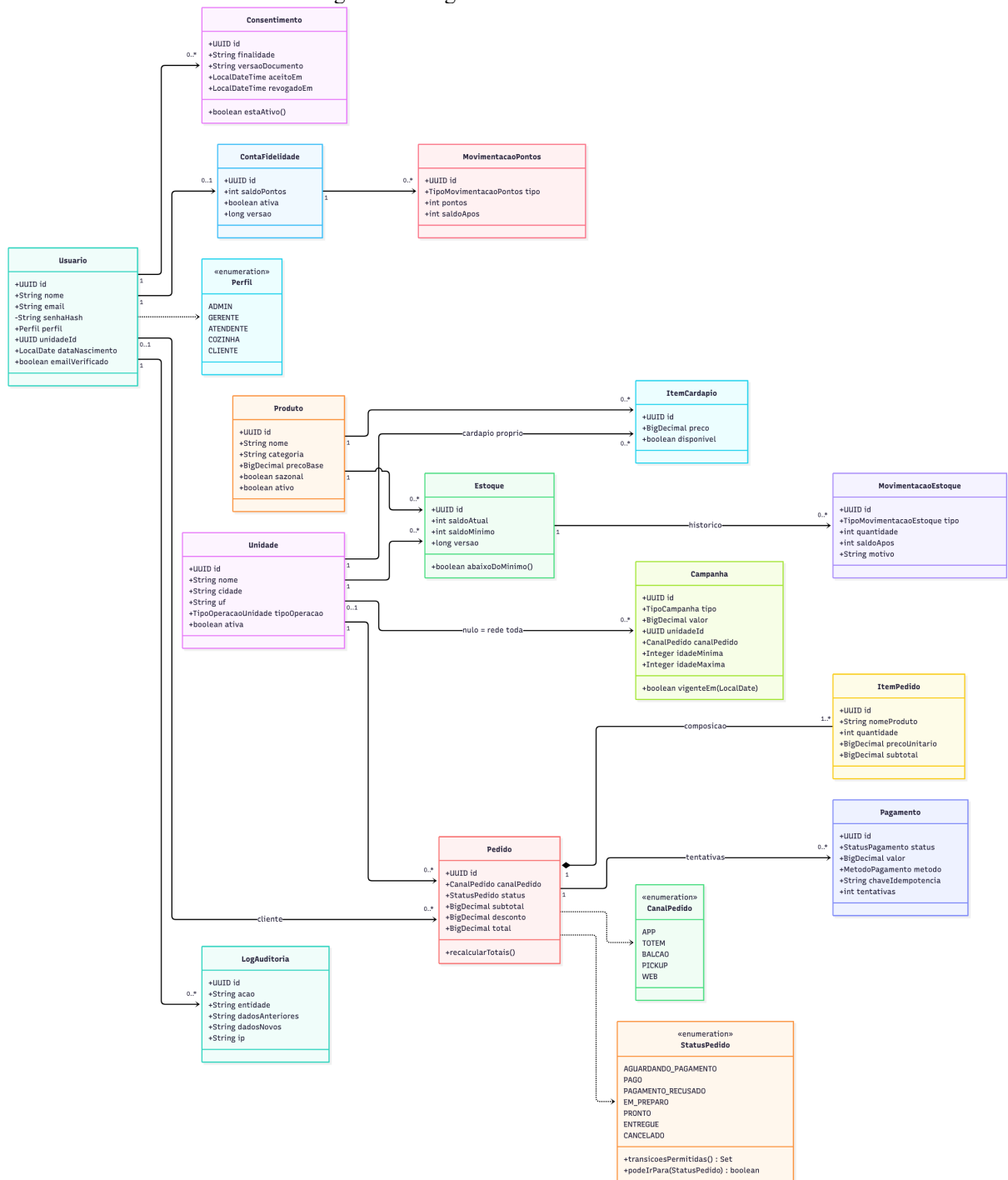
Tipo	Aplicação
Verificação de domínio	Método de pagamento restrito aos valores previstos; quantidade de item maior que zero
Concorrência	Coluna de versão em estoque e em conta de fidelidade, para bloqueio otimista
Nulidade significativa	Cliente nulo no pedido indica venda de balcão; unidade nula na campanha indica alcance de rede inteira; data de revogação nula indica consentimento vigente

Fonte: Elaborado pelo autor (2026).

3.4 Diagrama de classes

A Figura 5 apresenta a visão estrutural do domínio, com as entidades, seus atributos principais e as enumerações que restringem os valores possíveis.

Figura 5 – Diagrama de classes do domínio



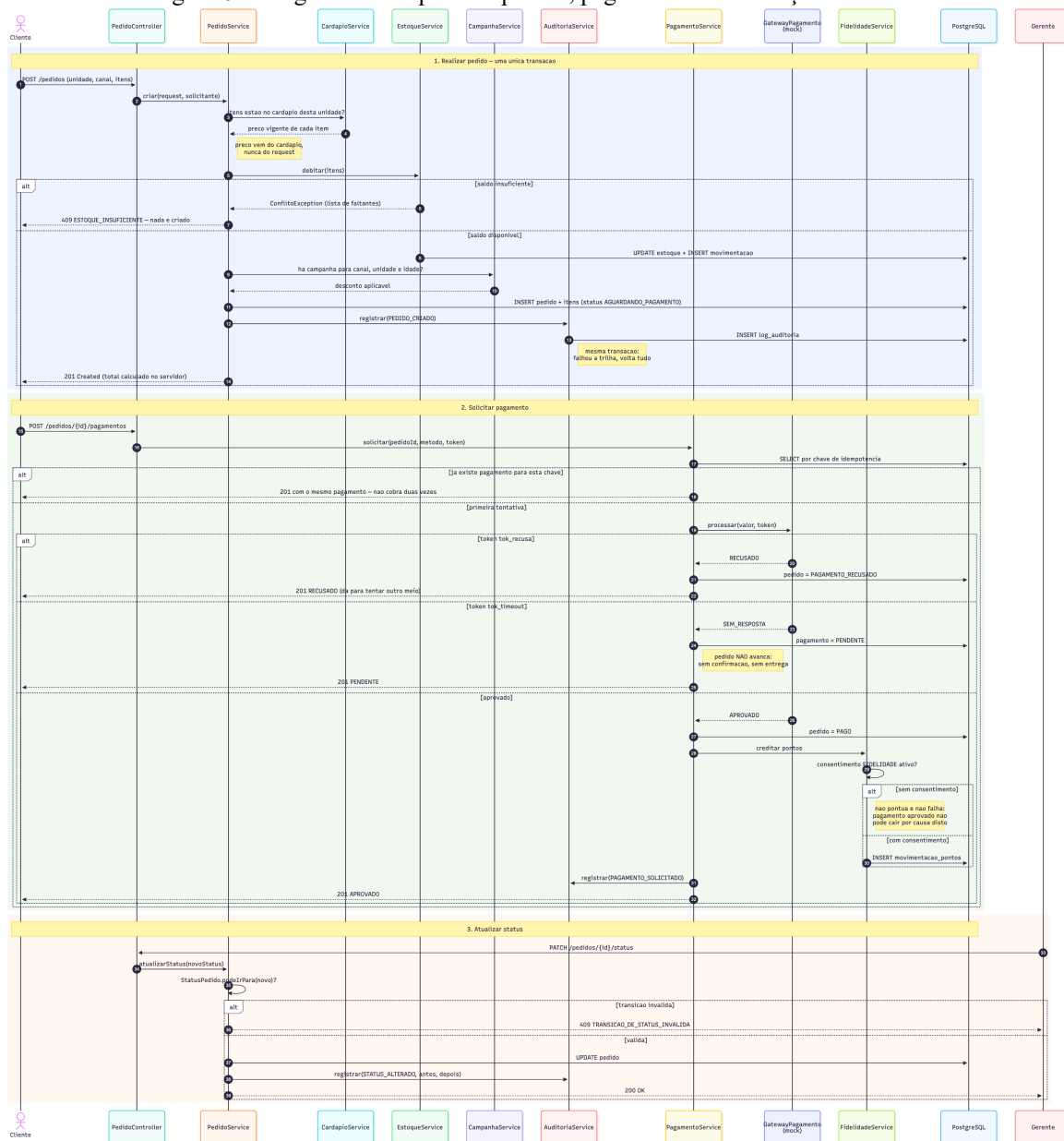
Fonte: Elaborado pelo autor (2026).

Vale destacar a enumeração de status do pedido. Ela não é apenas uma lista de valores: carrega o método que informa as transições permitidas a partir de cada estado. A regra da máquina de estados mora, portanto, no próprio domínio, e a camada de aplicação apenas pergunta a ela se a transição pretendida é válida — em vez de reimplementar a decisão em uma cadeia de condicionais.

3.5 Diagrama de sequência do fluxo crítico

A Figura 6 detalha, no tempo, a colaboração entre os componentes durante o fluxo crítico, dividida em três momentos: criação do pedido, solicitação de pagamento e atualização de status.

Figura 6 – Diagrama de sequência: pedido, pagamento e atualização de status



Fonte: Elaborado pelo autor (2026).

O diagrama evidencia duas decisões que não apareceriam na visão estrutural. A primeira é que o registro na trilha de auditoria ocorre dentro da mesma transação da ação: se a gravação da trilha falhar, a ação também é desfeita. Trilha que falha em silêncio não serve como prova,

e um pedido cancelado sem registro de quem o cancelou é exatamente a lacuna que a auditoria existe para fechar.

A segunda é o tratamento do crédito de pontos. Quando não há consentimento ativo, o sistema não pontua e tampouco interrompe o processamento: o pagamento permanece aprovado. Fazer um pagamento aprovado falhar por causa do programa de fidelidade prejudicaria o cliente mais do que a ausência do ponto.

4 ARQUITETURA

O projeto possui 95 arquivos e cerca de 8.200 linhas de código, distribuídos em nove pacotes. A organização segue o layout convencional do framework adotado, e esta seção explicita a correspondência com o vocabulário de camadas, a direção das dependências e a forma de verificá-la.

4.1 Organização em camadas

O Quadro 9 relaciona cada camada ao pacote correspondente no projeto e à sua responsabilidade.

Quadro 9 – Mapeamento das camadas para os pacotes do projeto

Camada	Pacote	Responsabilidade
API (Interface e Controllers)	controller, dto	Rotas, contratos de requisição e resposta, códigos de status
Application (Aplicação)	service	Casos de uso, orquestração e controle de transação
Domain (Domínio)	entity, domain	Entidades, enumerações, regras e invariantes do negócio
Infrastructure (Infraestrutura)	repository, config, security	Persistência, integrações e detalhes técnicos

Fonte: Elaborado pelo autor (2026).

A opção foi manter a nomenclatura convencional em vez de renomear os pacotes para o vocabulário de camadas. A decisão é deliberada e tem uma razão prática: qualquer pessoa familiarizada com o ecossistema abre os pacotes de controladores, serviços e repositórios e sabe de imediato o que esperar de cada um, enquanto um pacote nomeado por camada obrigaria o leitor a descobrir antes a convenção adotada localmente. Como o objetivo do requisito é clareza de responsabilidades, renomear produziria o efeito oposto ao pretendido.

4.2 Direção das dependências

A regra que organiza o projeto é única: a dependência aponta para dentro. A camada externa conhece a interna, nunca o contrário. O domínio, no centro, não depende de nenhuma outra. O Quadro 10 detalha o que essa regra proíbe e por quê.

Quadro 10 – Restrições de dependência entre camadas

Restrição	Motivo
A camada de API não acessa repositórios	Pular o serviço retiraria a regra de negócio e o controle de transação do lugar onde são testados
A camada de API não conhece entidades	Entidade é formato de banco. Devolvê-la diretamente vaza coluna que ninguém pediu, como o hash da senha, e amarra o contrato da API ao esquema: renomear uma coluna viraria mudança de contrato
Os objetos de transporte não conhecem entidades	Mesma razão, pelo lado oposto: o contrato precisa poder mudar sem que o banco mude
A camada de aplicação não conhece o protocolo HTTP	Serviço que conhece o objeto de requisição não pode ser reaproveitado fora de uma chamada web
O domínio não depende de camada alguma	É o centro da dependência. Se o domínio depende da infraestrutura, deixa de ser domínio

Fonte: Elaborado pelo autor (2026).

A separação não é apenas declarada: ela é verificável por inspeção das importações do código-fonte. O Quadro 11 apresenta a medição realizada sobre a versão entregue.

Quadro 11 – Verificação da separação entre camadas

Medição	Resultado
Arquivos que acessam repositórios na camada de API	0
Arquivos que importam entidades na camada de API	0
Objetos de transporte que importam entidades	0
Serviços que importam classes do protocolo HTTP	0
Estruturas condicionais ou de repetição nos controladores	0
Entidades devolvidas diretamente por um controlador	0

Fonte: Elaborado pelo autor (2026).

Existe uma exceção, e ela é consciente. O componente responsável por emitir o token de autenticação recebe a entidade do usuário, sendo o único ponto fora das camadas de aplicação e de persistência que a conhece. A alternativa seria passar separadamente os três valores que o token utiliza: identificador, perfil e unidade. Ela foi recusada porque identificador e unidade são ambos do mesmo tipo e ficariam adjacentes na assinatura do método: trocar um pelo outro na chamada compilaria, passaria nos testes, e produziria um token afirmando que a pessoa administra outra loja. Entre uma exceção documentada e um parâmetro fácil de inverter, a exceção é o risco menor.

4.3 Localização das regras de negócio

A coerência entre a modelagem, as regras e os endpoints é verificável pelo Quadro 12, que indica o componente responsável por cada regra de negócio do fluxo crítico.

Quadro 12 – Componente responsável por cada regra de negócio

Regra	Componente
RN01	CardapioService e PedidoService
RN02	PedidoService
RN03	PedidoService
RN04	EstoqueService
RN05	Validação declarativa nos objetos de transporte, com tratamento no manipulador global de exceções
RN06	PedidoService
RN07	PagamentoService
RN08	Enumeração StatusPedido, consultada por PedidoService
RN09	PedidoService e EstoqueService
RN10	FidelidadeService e ConsentimentoService
RN11	FidelidadeService
RN12	PedidoService
RN13	AuditoriaService, acionado pelos demais serviços
RN14	PagamentoService

Fonte: Elaborado pelo autor (2026).

Chama atenção a regra de transição de status, que não reside em um serviço, e sim na própria enumeração do domínio. É ela que conhece os destinos válidos a partir de cada estado, e a camada de aplicação apenas consulta se a transição pretendida é permitida. Regra de domínio no domínio: o serviço orquestra, não decide.

4.4 Portas para integrações externas

As duas integrações com serviços externos — envio de mensagem eletrônica e processamento de pagamento — são declaradas como interfaces no próprio domínio, e não como chamadas diretas a bibliotecas de terceiros. O domínio define o que precisa; quem cumpre fica do lado de fora.

Na versão entregue existem duas implementações de desenvolvimento: uma que registra as mensagens no log em vez de enviá-las, e outra que simula o serviço de pagamento com desfechos determinísticos. Substituir qualquer uma por um provedor real significa escrever outra implementação da mesma interface, sem alterar uma linha sequer dos serviços de pedido, de pagamento ou de verificação de cadastro. O custo de uma decisão que hoje é provisória fica, portanto, contido.

5 API E ENDPOINTS

A API expõe 30 rotas, que somam 38 operações distribuídas em onze recursos. Esta seção apresenta a visão geral do contrato, as convenções adotadas, o detalhamento das operações do fluxo crítico e o padrão de erro aplicado a todas as falhas.

5.1 Visão geral por recurso

O Quadro 13 relaciona as operações disponíveis, agrupadas por recurso.

Quadro 13 – Operações da API por recurso

Recurso	Oper.	Rotas
Autenticação	3	POST /auth/login · POST /auth/refresh · POST /auth/logout
Usuários	5	POST /usuarios · POST /usuarios/verificacao · POST /usuarios/verificacao/reenvio · POST /usuarios/operadores · GET /usuarios/me
Unidades	4	GET /unidades · GET /unidades/{id} · POST /unidades · PUT /unidades/{id}
Produtos	5	GET /produtos · GET /produtos/{id} · POST /produtos · PUT /produtos/{id} · DELETE /produtos/{id}
Cardápio	3	GET /unidades/{id}/cardapio · PUT /unidades/{id}/cardapio/{produtoId} · DELETE /unidades/{id}/cardapio/{produtoId}
Estoque	3	GET /unidades/{id}/estoque · POST /unidades/{id}/estoque/movimentacoes · GET /unidades/{id}/estoque/{produtoId}/movimentacoes
Pedidos	5	POST /pedidos · GET /pedidos · GET /pedidos/{id} · PATCH /pedidos/{id}/status · POST /pedidos/{id}/cancelamento
Pagamentos	3	POST /pedidos/{id}/pagamentos · GET /pagamentos/{id} · POST /pagamentos/callback
Fidelidade	3	GET /fidelidade/saldo · GET /fidelidade/extrato · POST /fidelidade/resgates
Consentimentos	3	POST /consentimentos · GET /consentimentos · DELETE /consentimentos/{id}
Auditoria	1	GET /auditoria

Fonte: Elaborado pelo autor (2026).

5.2 Convenções do contrato

As convenções do Quadro 14 valem para toda a API, de modo que conhecer um recurso seja suficiente para prever o comportamento dos demais.

Quadro 14 – Convenções adotadas no contrato da API

Convenção	Aplicação
Nomes no plural	Os recursos são nomeados no plural: /produtos, /pedidos, /unidades
Identificador no caminho	O identificador do recurso vai no caminho: /produtos/{produtoId}

Convenção	Aplicação
Subrecurso aninhado	O que só existe no contexto de outro recurso é aninhado: /unidades/{id}/estoque
Paginação obrigatória	Listagens aceitam page, começando em 1, e limit, com máximo de 100
Envelope de página	Toda listagem devolve conteúdo, página, limite, totalItens, totalPaginas, primeira e última
Autenticação	Cabeçalho Authorization com esquema Bearer, exceto nas rotas públicas
Rastreabilidade	Cabeçalho X-Request-Id aceito na entrada e devolvido na resposta e no corpo do erro

Fonte: Elaborado pelo autor (2026).

5.3 Detalhamento das operações do fluxo crítico

As três operações a seguir compõem o fluxo crítico e estão detalhadas no formato completo. As demais seguem as mesmas convenções e podem ser consultadas na documentação interativa, conforme indicado na subseção 5.5.

Quadro 15 – Operação criar pedido

Campo	Conteúdo
Finalidade	Criar um pedido na unidade informada, debitando o estoque e calculando o total.
Método e rota	POST /pedidos
Autenticação	Obrigatória. Perfis cliente e atendente.
Parâmetros	Nenhum parâmetro de caminho ou de consulta.
Corpo da requisição	<pre>{ "unidadeId": "10000000-0000-0000-0000-000000000001", "canalPedido": "APP", "itens": [{ "produtoId": "30000000-0000-0000-0000-000000000001", "quantidade": 2 }] }</pre>
Resposta 201	<pre>{ "id": "9690f527-c547-4a7a-b579-fec75154835c", "unidadeId": "10000000-0000-0000-0000-000000000001", "clienteId": "20000000-0000-0000-0000-000000000005", "canalPedido": "APP", "status": "AGUARDANDO_PAGAMENTO", "itens": [{ "produtoId": "30000000-0000-0000-0000-000000000001", "nome": "Tapioca de queijo coalho", "quantidade": 2, "precoUnitario": 12.90, "subtotal": 25.80 }], "subtotal": 25.80, "desconto": 2.58, "total": 23.22, "campanhaAplicada": "Primeira compra no app", }</pre>

Campo	Conteúdo
	<pre>"proximosStatus": ["PAGO", "PAGAMENTO_RECUSADO", "CANCELADO"], "criadoEm": "2026-10-03T16:26:24.031109", "atualizadoEm": "2026-10-03T16:26:24.031109" }</pre>
Códigos possíveis	201, 400, 401, 403, 404, 409, 422
Observação	O corpo não informa preço. O servidor lê o valor vigente no cardápio daquela unidade e o congela no item. O desconto decorre da campanha aplicável ao canal, à unidade e à faixa etária do cliente.

Fonte: Elaborado pelo autor (2026).

Quadro 16 – Operação solicitar pagamento

Campo	Conteúdo
Finalidade	Solicitar a cobrança de um pedido ao serviço externo de pagamento.
Método e rota	POST /pedidos/{pedidoId}/pagamentos
Autenticação	Obrigatória. Perfis cliente e atendente.
Parâmetros	Caminho: pedidoId, identificador do pedido.
Corpo da requisição	<pre>{ "metodo": "PIX", "tokenPagamento": "tok_ok_123" }</pre>
Resposta 201	<pre>{ "id": "38d64664-e108-422f-bcdd-006070718f28", "pedidoId": "9690f527-c547-4a7a-b579-fec75154835c", "status": "APROVADO", "valor": 23.22, "metodo": "PIX", "idTransacaoExterna": "mock_d415713d-2d0b-400f-b0a8-6c04935f1d12", "mensagem": "Pagamento aprovado.", "statusPedido": "PAGO", "tentativas": 1, "criadoEm": "2026-10-03T16:26:24.053250" }</pre>
Códigos possíveis	201, 400, 401, 403, 404, 409, 422
Observação	A operação é idempotente. O campo opcional chaveIdempotencia permite ao cliente reenviar a mesma solicitação sem gerar nova cobrança: o sistema devolve o pagamento já existente. Quando o serviço externo não responde dentro do tempo limite, o pagamento permanece pendente e o pedido não avança.

Fonte: Elaborado pelo autor (2026).

Quadro 17 – Operação atualizar status do pedido

Campo	Conteúdo
Finalidade	Avançar o pedido para o próximo estado do fluxo de preparo.
Método e rota	PATCH /pedidos/{pedidoId}/status
Autenticação	Obrigatória. Perfis gerente, atendente e cozinha, conforme a transição pretendida.
Parâmetros	Caminho: pedidoId, identificador do pedido.
Corpo da requisição	<pre>{ "status": "EM_PREPARO" }</pre>

Campo	Conteúdo
	}
Resposta 200	O pedido completo, no mesmo formato da criação, com o status atualizado e a nova lista de transições possíveis no campo proximosStatus.
Códigos possíveis	200, 400, 401, 403, 404, 409, 422
Observação	A transição é validada pela máquina de estados do domínio. Um destino não previsto a partir do estado atual resulta em conflito, com a indicação dos destinos válidos. Estados finais informam que não admitem mudança.

Fonte: Elaborado pelo autor (2026).

5.4 Padrão de erro

Toda falha, qualquer que seja a rota ou a causa, responde no mesmo envelope. O cliente trata um único formato, em vez de um por recurso.

```
{
  "error": "ESTOQUE_INSUFICIENTE",
  "message": "Não há quantidade suficiente para um ou mais itens.",
  "details": [
    { "field": "itens[Bolo de rolo].quantidade", "issue": "pedido: 999,
disponivel: 22" }
  ],
  "timestamp": "2026-10-03T16:26:24.153105Z",
  "path": "/pedidos",
  "requestId": "a594dea3-da90-4531-b1af-fbc2a64f43f5"
}
```

O campo error é um código estável, destinado ao tratamento programático; message destina-se à leitura humana; details enumera, campo a campo, o que impede a requisição de prosseguir. O campo requestId reproduz o valor recebido no cabeçalho X-Request-Id ou, na ausência dele, gera um novo. O mesmo identificador vai para o cabeçalho da resposta e para o registro de log, o que permite localizar no log a requisição exata a partir do identificador que o usuário informa.

O Quadro 18 apresenta os códigos de status utilizados e o significado atribuído a cada um.

Quadro 18 – Códigos de status HTTP utilizados

Código	Significado
200	Requisição bem-sucedida com corpo de resposta
201	Recurso criado
204	Requisição bem-sucedida sem corpo de resposta
400	O corpo não corresponde ao contrato: campo obrigatório ausente, enumeração inexistente, identificador mal formado

Código	Significado
401	Requisição não autenticada; também aplicado a rota inexistente quando não há credencial
403	Autenticado, porém sem permissão para a operação
404	Recurso inexistente, ou não visível para quem o solicitou
405	Método não previsto para a rota
409	Conflito com o estado atual: saldo insuficiente, pedido já pago, transição inválida
415	Tipo de mídia não suportado
422	O corpo corresponde ao contrato, mas o valor é inválido: senha fraca, preço negativo

Fonte: Elaborado pelo autor (2026).

A distinção entre 400 e 422 é intencional: 400 significa que o corpo não foi compreendido, e 422, que foi compreendido e não é aceito. Na linguagem adotada, campo obrigatório ausente falha na desserialização antes da validação declarativa, o que produz naturalmente o primeiro caso.

Também é deliberado responder 401, e não 404, para rota inexistente quando não há credencial. Responder 404 informaria a quem não se autenticou quais caminhos existem e quais não, entregando o mapa da API a qualquer visitante.

Os códigos de erro previstos constam do Quadro 19.

Quadro 19 – Códigos de erro da API

Código	HTTP	Situação
NAO_AUTENTICADO	401	Credencial ausente, inválida ou expirada
SEM_PERMISSAO	403	Perfil sem autorização para a operação
REQUISICAO_INVALIDA	400	Corpo em desacordo com o contrato
VALIDACAO	422	Valor inválido em campo que atende ao contrato
UNIDADE_NAO_ENCONTRADA	404	Unidade inexistente
PRODUTO_NAO_ENCONTRADO	404	Produto inexistente
PEDIDO_NAO_ENCONTRADO	404	Pedido inexistente ou fora do escopo de visão
EMAIL_JA_CADASTRADO	409	Endereço já em uso, em rota autenticada
EMAIL_NAO_VERIFICADO	403	Conta criada, porém sem confirmação de endereço
CODIGO_VERIFICACAO_INVALIDO	400	Código de confirmação incorreto ou expirado
ESTOQUE_INSUFICIENTE	409	Saldo indisponível para um ou mais itens
PRODUTO_FORA_DO_CARDAPIO	422	Produto ausente do cardápio daquela unidade
TRANSICAO_DE_STATUS_INVALIDA	409	Transição não prevista na máquina de estados
PEDIDO_JA_PAGO	409	Nova cobrança para pedido já quitado
PONTOS_INSUFICIENTES	409	Resgate maior que o saldo disponível
METODO_NAO_PERMITIDO	405	Método HTTP não previsto para a rota
TOKEN_INVALIDO	401	Token de renovação já utilizado ou revogado

Fonte: Elaborado pelo autor (2026).

5.5 Documentação interativa e coleção de testes

A especificação OpenAPI é gerada a partir do próprio código, o que garante que descreva as rotas efetivamente implementadas e não uma intenção desatualizada. Com a aplicação em execução, a interface interativa fica disponível em `/swagger-ui.html` e o documento bruto em `/v3/api-docs`. A interface permite autenticar-se uma vez e exercitar qualquer rota protegida pelo navegador.

O repositório traz ainda a especificação exportada e a coleção de testes, ambas versionadas, cujo uso é detalhado na Seção 7.

6 LGPD, PRIVACIDADE E SEGURANÇA

A Lei Geral de Proteção de Dados foi tratada como requisito de projeto, e não como declaração acessória. Esta seção identifica os dados pessoais tratados, a finalidade e a base legal de cada um, os controles técnicos aplicados e as decisões de segurança que não correspondem ao caminho mais curto.

6.1 Dados pessoais tratados

O Quadro 20 relaciona os dados pessoais tratados pelo sistema. A separação por base legal é o que determina o comportamento: o que se apoia em execução de contrato é necessário para a venda acontecer; o que se apoia em consentimento pode ser recusado pelo titular sem que ele perca o acesso ao serviço.

Quadro 20 – Dados pessoais, finalidade e base legal

Dado	Finalidade	Base legal
Nome e endereço eletrônico	Identificação e autenticação do titular	Execução de contrato, art. 7º, inciso V
Senha, armazenada como resumo criptográfico	Autenticação	Execução de contrato, art. 7º, inciso V
Histórico de pedidos	Operação da venda e acompanhamento do preparo	Execução de contrato, art. 7º, inciso V
Data de nascimento	Segmentação de campanha por faixa etária	Consentimento, art. 7º, inciso I
Perfil de consumo e acúmulo de pontos	Programa de fidelidade	Consentimento, art. 7º, inciso I
Endereço de rede de origem	Trilha de auditoria e segurança	Legítimo interesse, art. 7º, inciso IX

Fonte: Elaborado pelo autor (2026).

A consequência prática dessa separação aparece no programa de fidelidade. Acumular pontos exige saber quem é o cliente e o que ele consome, o que caracteriza tratamento apoiado em consentimento. Sem consentimento ativo, o pedido é pago normalmente e simplesmente não gera pontos: o titular que recusa não é impedido de comprar.

6.2 Consentimento e revogação

O consentimento é registrado por finalidade, e não de forma global. Cada registro guarda a finalidade, a versão do documento aceito, o instante do aceite e o endereço de rede de origem.

Versionar o documento é o que permite demonstrar, no futuro, a que texto exatamente o titular aderiu.

A revogação não apaga o registro: preenche a data de revogação e preserva a linha. O histórico é a prova de que o tratamento foi legítimo enquanto durou, e apagá-lo destruiria justamente a evidência que a lei exige que o controlador seja capaz de apresentar. A revogação da finalidade de fidelidade desativa a conta de pontos de imediato; o saldo acumulado permanece, mas nada novo é creditado.

6.3 Controles técnicos

O Quadro 21 apresenta os controles implementados.

Quadro 21 – Controles técnicos de segurança

Controle	Implementação
Armazenamento de senha	Resumo criptográfico com função de derivação lenta e sal próprio por senha. Não há forma de recuperar a senha original a partir do armazenado
Autenticação	Token assinado com validade de 15 minutos, acompanhado de token de renovação com rotação: usado uma vez, deixa de valer
Autorização	Cinco perfis, verificados por endpoint. A configuração nega por padrão: rota nova nasce protegida e só se torna pública por liberação explícita
Escopo por unidade	O gerente administra apenas a própria loja. A tentativa de alterar outra unidade é recusada por falta de permissão
Escopo por titular	O cliente enxerga somente os próprios pedidos. Pedido de terceiro responde como inexistente, e não como proibido
Minimização nas respostas	Objetos de saída dedicados. O resumo da senha não possui campo correspondente em nenhuma resposta da API
Rastreabilidade	Identificador de requisição propagado entre resposta, corpo de erro e registro de log
Proteção do retorno de pagamento	A rota pública de confirmação exige segredo compartilhado, comparado em tempo constante

Fonte: Elaborado pelo autor (2026).

6.4 Decisões de segurança

Seis decisões merecem registro por não corresponderem ao comportamento mais imediato. Três delas tratam do mesmo risco, a enumeração de titulares: a possibilidade de descobrir quem possui conta no sistema testando endereços e observando as respostas. O Quadro 22 as apresenta.

Quadro 22 – Decisões de segurança adotadas

Decisão	Justificativa
O cadastro não revela se um endereço já possui conta	A criação de conta responde sempre da mesma forma, exista ou não o endereço informado. Devolver conflito para endereço duplicado seria mais informativo e também seria um oráculo: bastaria testar endereços para descobrir quem é cliente da rede. Para que a medida não impeça o legítimo titular de se cadastrar, a conta só se completa após a confirmação do código enviado ao endereço
A autenticação não revela existência pela mensagem nem pelo tempo de resposta	A primeira implementação encerrava a verificação mais cedo quando o usuário não existia, sem calcular o resumo da senha. A diferença de tempo, medida em cerca de 58 milissegundos, respondia à pergunta que a mensagem genérica se recusava a responder. A correção faz ambos os caminhos custarem o mesmo
A confirmação de endereço não revela contas já verificadas	A confirmação responde sem conteúdo quando a conta já está verificada, pois confirmar duas vezes não é erro. A primeira versão, porém, respondia assim para qualquer código: bastava enviar um código arbitrário e ler o status para saber se o endereço existe e está ativo. A correção condiciona essa resposta à apresentação do código correto; nos demais casos, a resposta é idêntica à de um endereço inexistente
Pedido de terceiro responde como inexistente	A recusa por falta de permissão confirmaria que o pedido existe. Para quem não pode vê-lo, ele simplesmente não existe
A documentação interativa não fica disponível em produção	O perfil de produção desabilita a interface de exploração da API. Ela é ferramenta de desenvolvimento e de avaliação; publicá-la entrega o mapa completo das rotas a qualquer visitante
Nenhum segredo possui valor padrão em produção	No perfil de produção, a ausência de qualquer variável de ambiente sensível impede a aplicação de iniciar. Subir com segredo de exemplo é pior do que não subir

Fonte: Elaborado pelo autor (2026).

A terceira decisão do quadro merece nota de método: ela não foi encontrada por revisão de código, e sim pela execução completa do roteiro de validação contra uma base recém-criada. O cenário esperava recusa e recebeu resposta de sucesso. O caso ilustra por que a verificação executável tem valor distinto da leitura do código: a falha estava em um caminho que a leitura considerava correto, porque confirmar duas vezes realmente não deveria ser erro.

6.5 Trilha de auditoria

Nove ações sensíveis geram registro, conforme o Quadro 23. Cada registro guarda a ação, o tipo e o identificador do recurso afetado, o estado anterior, o estado novo, o autor e o endereço de rede de origem.

Quadro 23 – Ações registradas na trilha de auditoria

Ação	Recurso afetado
PEDIDO_CRIADO	Pedido
PEDIDO_CANCELADO	Pedido
STATUS_ALTERADO	Pedido

Ação	Recurso afetado
ESTOQUE_MOVIMENTADO	Estoque
PAGAMENTO_SOLICITADO	Pagamento
PONTOS_ACUMULADOS	Fidelidade
PONTOS_RESGATADOS	Fidelidade
CONSENTIMENTO_REGISTRADO	Consentimento
CONSENTIMENTO_REVOGADO	Consentimento

Fonte: Elaborado pelo autor (2026).

Três características sustentam a trilha como prova.

A primeira é a transacionalidade: o registro ocorre na mesma transação da ação. Se a gravação da trilha falhar, a ação é desfeita. Trilha que falha em silêncio não serve como prova, e um pedido cancelado sem registro de quem o cancelou é exatamente a lacuna que a auditoria existe para fechar.

A segunda é a ausência de rota de escrita ou de exclusão. A API expõe apenas consulta, e nenhuma rotina do sistema altera ou remove linha dessa tabela. Sem endpoint que a modifique, não há como adulterar a prova pela interface.

A terceira é a restrição de leitura ao perfil administrador. A trilha revela o que cada pessoa fez na rede e, por isso, é ela própria um dado sensível: liberá-la ao gerente daria a ele visão da operação das demais lojas.

6.6 Limitações conhecidas

Duas medidas pertinentes ao tema não foram implementadas, e o registro da lacuna é preferível à omissão.

A limitação de taxa de requisições nos endpoints públicos tem as bibliotecas declaradas no projeto, porém o filtro não foi conectado. A ausência não cria falha de autenticação ou de autorização, mas deixa as rotas públicas sem proteção contra tentativa automatizada em volume.

A anonimização após o prazo de retenção depende de uma política que o caso fictício não define. Implementar um prazo arbitrário significaria inventar uma regra de negócio, o que seria menos honesto do que declarar a decisão pendente.

7 ENTREGA TÉCNICA

A entrega compreende o código-fonte em repositório público, a documentação de execução, a especificação da API e a coleção de testes reproduzível. Esta seção indica onde encontrar cada item e como colocar a aplicação em funcionamento.

7.1 Repositório

O código está disponível publicamente no endereço <https://github.com/geanbrandao/raizes-do-nordeste-api>, com histórico de commits que acompanha a evolução do trabalho. A organização do repositório consta do Quadro 24.

Quadro 24 – Organização do repositório

Caminho	Conteúdo
src/main/kotlin	Código-fonte, organizado nos nove pacotes descritos na Seção 4
src/main/resources/db/migration	Dezoito migrações versionadas, incluindo a carga inicial de dados
src/test/kotlin	Testes automatizados
postman	Coleção de testes, ambiente e especificação exportada
docs/diagramas	Os seis diagramas deste documento, em formato vetorial e com o código-fonte
README.md	Instruções de execução, decisões técnicas e evidências
docker-compose.yml e Dockerfile	Definição do ambiente
.env.example	Variáveis de ambiente com valores de exemplo

Fonte: Elaborado pelo autor (2026).

7.2 Requisitos e execução

A execução exige apenas um ambiente de contêineres. A compilação ocorre dentro do contêiner, de modo que não é necessário instalar linguagem, ferramenta de construção ou banco de dados na máquina. As portas utilizadas são a 8080, para a aplicação, e a 5432, para o banco.

Quadro 25 – Passos para execução

Passo	Ação
1	Clonar o repositório
2	Executar <code>docker compose down -v</code> e, em seguida, <code>docker compose up --build</code>
3	Aguardar o registro indicando a aplicação iniciada no log do contêiner
4	Confirmar a resposta de <code>/actuator/health</code>

Fonte: Elaborado pelo autor (2026).

A opção que remove o volume antes de iniciar não é zelo excessivo: as seções da coleção de testes alteram preço, estoque e cadastro de maneira deliberada, e uma base reaproveitada faz os valores esperados deixarem de corresponder, sem que exista defeito na aplicação.

O esquema do banco e a carga inicial são aplicados automaticamente na inicialização, por migrações versionadas. Não há passo manual de criação de tabelas. O arquivo de exemplo de variáveis de ambiente acompanha o repositório; para desenvolvimento, os valores padrão já estão definidos e nada precisa ser configurado.

7.3 Documentação da API

A especificação é gerada a partir do código, de modo que descreve as rotas efetivamente implementadas. Com a aplicação em execução, a interface interativa permite autenticar-se uma vez, pelo botão de autorização, e exercitar qualquer rota protegida diretamente pelo navegador. Em produção a interface é desabilitada, conforme justificado na Seção 6.

7.4 Coleção de testes

A coleção acompanha o repositório e foi concebida para ser executada sem preparação manual. As pastas estão na ordem de execução, e cada requisição guarda automaticamente o que a seguinte necessita: os tokens dos quatro perfis, os identificadores de pedido, de pagamento e de consentimento. Não há valor a copiar entre requisições.

A execução completa dispara 202 verificações automáticas. A coleção foi confirmada tanto na ferramenta de origem quanto em cliente alternativo, e pode ainda ser executada por linha de comando, sem interface gráfica.

7.5 Evidências

O Quadro 26 relaciona os endereços e caminhos de cada item da entrega.

Quadro 26 – Evidências da entrega

Item	Endereço ou caminho
Repositório	https://github.com/geanbrandao/raizes-do-nordeste-api
Documentação interativa	http://localhost:8080/swagger-ui.html , com a aplicação em execução
Especificação bruta	http://localhost:8080/v3/api-docs
Especificação versionada	postman/openapi.json

Item	Endereço ou caminho
Coleção de testes	postman/raizes-do-nordeste.postman_collection.json
Ambiente da coleção	postman/raizes-do-nordeste.postman_environment.json
Roteiro de validação manual	VALIDACAO.md
Decisões de arquitetura	ARQUITETURA.md
Decisões de segurança	DECISOES-SEGURANCA.md

Fonte: Elaborado pelo autor (2026).

8 PLANO DE TESTES

A validação ocorre em dois níveis complementares. O primeiro é automatizado e executa sem banco de dados externo, usando um banco em memória em modo de compatibilidade e aplicando as mesmas migrações da aplicação: uma divergência entre migração, entidade e carga inicial quebra o teste, e não a inicialização em produção. O segundo é a coleção de requisições, que exercita a aplicação em funcionamento e serve de evidência reproduzível para terceiros.

8.1 Cenários

O Quadro 27 apresenta os cenários de teste, com a identificação da requisição correspondente na coleção. São catorze cenários, sendo seis positivos e oito negativos, acima do mínimo exigido.

Quadro 27 – Cenários de teste

Identificação	Endpoint	Pré-condição	Entrada	Resultado esperado
T01 Login válido	POST /auth/login	Usuário do seed existente	Endereço e senha	200 com token de acesso e validade de 900 segundos
T02 Acesso sem token	GET /usuarios/me	Nenhuma	Sem cabeçalho de autorização	401 com NAO_AUTENTICADO
T03 Perfil sem permissão	POST /unidades	Autenticado como cliente	Dados de uma unidade	403 com SEM_PERMISSAO
T04 Pedido sem canal	POST /pedidos	Autenticado como cliente	Unidade e itens, sem canalPedido	400 com REQUISICAO_INVALIDA, apontando o campo
T05 Quantidade negativa	POST /pedidos	Autenticado como cliente	Item com quantidade igual a menos três	422 com VALIDACAO, apontando o item
T06 Pedido com itens válidos	POST /pedidos	Cardápio e estoque disponíveis na unidade	Unidade, canal e dois itens	201 com total calculado pelo servidor e preço congelado
T07 Produto inexistente	PUT /unidades/{id}/cardapio/{produtoId}	Autenticado como administrador	Identificador de produto inexistente	404 com PRODUTO_NAO_ENCONTRADO
T08 Estoque insuficiente	POST /pedidos	Produto com saldo menor que o pedido	Item com quantidade acima do saldo	409 com ESTOQUE_INSUFICIENTE e lista do que faltou

Identificação	Endpoint	Pré-condição	Entrada	Resultado esperado
T09 Pagamento aprovado	POST /pedidos/{id}/pagamentos	Pedido aguardando pagamento	Método e identificador de cobrança	201 com APROVADO e pedido em PAGO
T10 Pagamento recusado	POST /pedidos/{id}/pagamentos	Pedido aguardando pagamento	Identificador que força recusa	201 com RECUSADO e pedido em PAGAMENTO_RECUSADO
T11 Cardápio da unidade	GET /unidades/{id}/cardapio	Carga inicial aplicada	Identificador da unidade	200 com os dez itens da unidade comple
T12 Filtro por canal	GET /pedidos	Existir pedido no canal consultado	Parâmetro canalPedido igual a TOTEM	200 contendo apenas pedidos daquele canal
T13 Transição inválida	PATCH /pedidos/{id}/status	Pedido em estado final	Status de destino não permitido	409 com TRANSICAO_DE_STATUS_INVALID
T14 Auditoria da criação	GET /auditoria	Autenticado como administrador, com pedido criado	Filtro por entidade igual a PEDIDO	200 contendo registro PEDIDO_CRIAD com autor e origem

Fonte: Elaborado pelo autor (2026).

A coluna de evidência indica a pasta da coleção onde a requisição se encontra. O nome de cada requisição inicia pela identificação do cenário, de modo que a correspondência seja imediata.

8.2 Cobertura

O Quadro 28 relaciona as áreas de cobertura exigidas aos cenários que as atendem.

Quadro 28 – Cobertura por área

Área	Cenários
Autenticação e autorização	T01, T02, T03
Validação de dados	T04, T05
Regras de negócio do fluxo principal	T06, T07, T08
Pagamento simulado e atualização de status	T09, T10, T13
Consulta e multicanalidade	T11, T12
Registro de auditoria	T14

Fonte: Elaborado pelo autor (2026).

Além dos catorze cenários, a coleção contém trinta casos negativos adicionais, que verificam o código de erro e o conteúdo do campo de detalhes para cada situação prevista na Seção 5.

8.3 Testes automatizados

Os testes automatizados cobrem o mesmo comportamento em nível de unidade e de integração, incluindo situações de difícil reprodução manual. Dois exemplos ilustram o critério adotado.

O primeiro verifica que a função de verificação de senha é invocada mesmo quando o endereço informado não existe. A propriedade a preservar é a equivalência de tempo entre os dois caminhos, e medi-la diretamente produziria um teste instável; verificar a invocação é estável e protege a mesma propriedade.

O segundo captura a saída de registro de log para confirmar que o código de confirmação de endereço não é escrito em log fora do ambiente de desenvolvimento. A ausência de um valor no log é difícil de verificar por inspeção e trivial de reintroduzir por descuido.

8.4 Resultados

O Quadro 29 apresenta os resultados da última execução, realizada contra banco de dados criado do zero.

Quadro 29 – Resultados da validação

Verificação	Resultado
Testes automatizados	196 executados, nenhuma falha
Requisições da coleção	93 executadas, nenhuma falha
Verificações da coleção	202 executadas, nenhuma falha
Cenários negativos na coleção	30, cada um conferindo o status e o código de erro
Migrações aplicadas	18, todas bem-sucedidas em banco criado do zero

Fonte: Elaborado pelo autor (2026).

Cabe registrar uma limitação do método. A execução da coleção sobre uma base já utilizada faz seis verificações falharem, porque as pastas de cadastro, cardápio e estoque alteram estado de maneira deliberada. Nenhuma dessas falhas indica defeito na aplicação, e a própria descrição da coleção enumera quais são e por quê. A recomendação de recriar o banco antes de cada execução decorre dessa característica.

9 CONCLUSÃO

O trabalho entregou uma API REST em funcionamento para a operação multiunidade e multicanal da rede, com 28 dos 32 requisitos funcionais implementados e os quatro restantes justificados na subseção 2.4. O critério de priorização foi o risco técnico: foram construídos primeiro os requisitos que concentram decisões difíceis, e adiados os que reaproveitam mecanismos já demonstrados.

A modelagem conecta-se diretamente ao que foi entregue. Os casos de uso da Figura 1 correspondem aos recursos do Quadro 13; as entidades do diagrama da Figura 4 sustentam cada endpoint descrito na Seção 5; e as regras de negócio do Quadro 3 têm componente responsável identificado no Quadro 12. A máquina de estados do pedido, representada no diagrama de classes, é a mesma que recusa transições inválidas em tempo de execução, porque reside na enumeração do domínio e não em uma reimplementação na camada de aplicação.

A integração de pagamento foi tratada como integração real, ainda que simulada. A dependência externa é declarada como interface no domínio, e a implementação de desenvolvimento produz os três desfechos que um provedor verdadeiro produz: aprovação, recusa e ausência de resposta. O terceiro caso é o que justifica o estado intermediário de pagamento pendente e a confirmação posterior por retorno assíncrono autenticado. Substituir o simulador por um provedor real significa escrever outra implementação da mesma interface, sem alterar os serviços de pedido ou de pagamento.

A padronização de erros, detalhada na Seção 5, garante que toda falha responda no mesmo envelope, com código estável para tratamento programático, mensagem legível e detalhamento campo a campo. A distinção entre corpo não compreendido e valor não aceito, assim como a decisão de não revelar a existência de rotas a quem não se autenticou, decorrem do mesmo princípio: a resposta de erro não deve informar mais do que o necessário.

Quanto à proteção de dados, três decisões se destacam por não corresponderem ao caminho mais curto, e estão detalhadas na Seção 6: a resposta uniforme no cadastro, a equivalência de tempo na autenticação e a condição imposta à confirmação de endereço. As duas primeiras foram projetadas; a terceira foi descoberta durante a validação executável, quando um cenário que esperava recusa recebeu resposta de sucesso. O programa de fidelidade, por depender de base legal de consentimento, não pontua quem não consentiu, sem que isso impeça a compra.

A validação, descrita na Seção 8, combina 196 testes automatizados e 93 requisições organizadas em coleção reproduzível, que disparam 202 verificações. O conjunto cobre as seis áreas exigidas, com catorze cenários identificados e rastreáveis até a requisição correspondente.

Entre as lições do processo, a mais relevante diz respeito ao valor da verificação executável. Três defeitos só apareceram quando o roteiro completo foi executado contra uma base criada do zero, e nenhum deles seria encontrado por leitura: dois estavam em caminhos que a leitura considerava corretos, e um era um documento de especificação inválido que a ferramenta de desenvolvimento aceitava por servir da mesma origem. Testes que passam não substituem a execução do caminho real.

Como evolução, a limitação de taxa nos endpoints públicos é a lacuna mais próxima de ser fechada, por já ter as bibliotecas declaradas. Em seguida viriam a política de retenção e anonimização, que depende de definição de negócio, e os relatórios consolidados para a matriz, cujos dados já estão modelados e acessíveis.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6023: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 6024: informação e documentação: numeração progressiva das seções de um documento: apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 10520: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 4 out. 2026.

EVANS, Eric. Domain-driven design: atacando as complexidades no coração do software. Rio de Janeiro: Alta Books, 2016.

FIELDING, Roy Thomas. Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado em Information and Computer Science) - University of California, Irvine, Irvine, 2000.

FOWLER, Martin. Padrões de arquitetura de aplicações corporativas. Porto Alegre: Bookman, 2006.

FOWLER, Martin. UML essencial: um breve guia para a linguagem padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

INTERNET ENGINEERING TASK FORCE. RFC 7519: JSON Web Token (JWT). [S. l.]: IETF, 2015. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519>. Acesso em: 4 out. 2026.

INTERNET ENGINEERING TASK FORCE. RFC 9110: HTTP semantics. [S. l.]: IETF, 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110>. Acesso em: 4 out. 2026.

JETBRAINS. Kotlin documentation. [S. l.], 2026. Disponível em: <https://kotlinlang.org/docs/home.html>. Acesso em: 4 out. 2026.

OPENAPI INITIATIVE. OpenAPI specification version 3.1.0. [S. l.], 2021. Disponível em: <https://spec.openapis.org/oas/v3.1.0>. Acesso em: 4 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 documentation. [S. l.], 2026. Disponível em: <https://www.postgresql.org/docs/16/index.html>. Acesso em: 4 out. 2026.

PROVOS, Niels; MAZIÈRES, David. A future-adaptable password scheme. In: USENIX ANNUAL TECHNICAL CONFERENCE, 1999, Monterey. Proceedings [...]. Berkeley: USENIX Association, 1999. p. 81-91.

REDGATE SOFTWARE. Flyway documentation. [S. l.], 2026. Disponível em: <https://documentation.red-gate.com/flyway>. Acesso em: 4 out. 2026.

VMWARE. Spring Boot reference documentation. [S. l.], 2026. Disponível em: <https://docs.spring.io/spring-boot/index.html>. Acesso em: 4 out. 2026.

VMWARE. Spring Security reference documentation. [S. l.], 2026. Disponível em: <https://docs.spring.io/spring-security/reference/index.html>. Acesso em: 4 out. 2026.